Reprise

Check what you already generated before you pay to generate it again. Backblaze Generative Media Hackathon 2026.

The problem

Generative media teams pay twice for the same asset. The same product shot, the same jingle, weeks apart, requested by people who had no way to know it already existed. The bucket holding those assets is treated as an archive. Reprise treats it as the opposite: an asset that appreciates, placed in the request path so that owning something is checked before paying for it again.

Decision model

```
prompt -> exact match?  ---- yes -> REUSE    serve from B2, $0, saving booked
            | no
          embed + score the library (cosine over gemini-embedding-001)
            |- similarity >= 0.97 -> REUSE    auto-serve
            |- 0.85 .. 0.97       -> REVIEW   candidate shown to a human; $0 until accepted
            '- below 0.85         -> GENERATE Genblaze pipeline -> B2 + provenance manifest
```

Modality, aspect ratio and style are hard constraints, not scored signals: they are substitutability requirements, and a 0.99 prompt match at the wrong aspect ratio is not a substitute. Similarity measures how alike two prompts read, not how alike two images look, which is exactly why the review band exists.

Measured policy

category	n	reuse	review	generate
exact repeat	6	6	0	0
equivalent phrasing	10	5	5	0
attribute swap (red to blue, mug to glass)	10	0	9	1
same domain, different ask	7	0	0	7
unrelated	5	0	0	5

0 of 17 dangerous pairs auto-reused. 0 of 10 non-exact equivalents regenerated. Highest attribute-swap similarity measured 0.968, just under the 0.97 auto line. That thin margin is published rather than hidden, and it is the reason attribute swaps land in front of a person instead of silently on a customer. CI recomputes every verdict from the stored similarities and fails the build if the README, the report, or the test count drifts from the data.

Backblaze B2 is the product's memory

- **Assets** content-addressed by digest via `KeyStrategy.CONTENT_ADDRESSABLE`, so identical bytes dedupe by construction.
- **Manifests** beside assets. The library is a projection of them, so there is no second database to drift.
- **Embedding sidecars** keyed by normalized-prompt hash: a library prompt is embedded once, however many runs reference it.
- **Decision ledger** written with Object Lock GOVERNANCE retention, computed at each write.
- **Serving** by short-lived presigned URL, with a prefix containment check so a foreign manifest cannot turn the serve path into a presigned-URL oracle.

```
reprise/
  assets/<sha[:2]>/<sha[2:4]>/<sha>.png
  manifests/<run_id>.json
  embeddings/<prompt-sha>.json
  ledger/<ts>-<seq>-<fp>-<nonce>.json
```

The savings scoreboard is recomputed from ledger objects on read. Nothing caches a number the ledger cannot back.

Genblaze does the orchestration

- Every generation is a `Pipeline` with `ObjectStorageSink(raise_on_failure=True)`: nothing enters the library without a sha256-bound, verifiable manifest.
- **Admission is gated on `Manifest.verify_hash()`**. A manifest whose payload no longer matches its canonical hash is refused entirely, so editing a stored run's prompt cannot point a popular prompt at bytes of someone else's choosing. Per-asset rules (succeeded steps, sha256-bound assets, prompted steps) then filter within a verified manifest.
- **A custom `SyncProvider`** for Gemini-native image models, written because every `imagen-*` tier answers 404 "no longer available to new users" on newly created keys.
- **A model fallback chain.** A slug in a provider catalog is not a slug you are entitled to call, so a 404 walks the chain and the manifest records the model that actually produced the bytes.
- Stock `ElevenLabsTTSProvider` for audio, behind the same interface.

Proof receipt

Every result carries the coordinates needed to check it independently: Genblaze run id, manifest key and its canonical hash, content-addressed asset key, sha256, producing provider and model, original cost, and the retention actually in force (read from the ledger's own state, so the UI cannot advertise a lock this instance does not write). These are values a viewer re-derives from the bucket, not a "verified" badge the app issues about itself.

Spend and trust posture

risk	control
Uncounted spend	Two daily budgets (generation, decision embeddings), both RESERVED before the provider call. An unwritable reservation counts as cap-exhausted (fail closed).
Free path billed	The gateway calls back immediately before its first billable embedding, so an exact repeat consumes no paid quota.
Forged savings	Acceptance requires an HMAC capability token minted with the review that offered it.
Replayed savings	Each offer is named by the token and can be accepted once; a replay gets 409.
Tampered provenance	Manifest hash gate on admission.
Leaked internals	Upstream errors return a correlation id; the real error is logged, never returned.
XSS via prompt text	Results are built as DOM nodes with textContent; no API value is ever concatenated into HTML.

Stated limitations

- Prompt similarity is not image similarity. An image-embedding second signal is the obvious next move, and would narrow the review band with engineering rather than by moving a number.
- Cap and replay checks are read-then-act, so a concurrent burst can overshoot by roughly the concurrency level. Reservations still land, so nothing goes uncounted.
- Demo-scale scans re-list the bucket per request; Genblaze's `ParquetSink` tables are the natural index seed.
- Object Lock is described precisely: a delete can hide a record behind a delete marker (tamper-evident, recoverable); it cannot destroy the version while retention holds.

Verification

`88 tests, ruff, mypy strict, an eval-freshness pin, and a long-dash scan, all in CI. Live probe outputs, including two probe corrections kept deliberately, are in docs/run-evidence.md.`

Live: reprise-murex.vercel.app · Source: github.com/OrionArchitekton/reprise
Upstream SDK feedback filed from this build: backblaze-labs/genblaze issues 205 and 206.